
Technical report for AIRI 2025

Volovikov Georgiy¹

Abstract

Many thanks to the AIRI team for providing this task. It was a wonderful experience for me. I managed to set up a pipeline for collecting data from environments, validating the model, and training it, as well as testing several hypotheses. I wasn't able to further train the model using GRPO or to launch VLA.

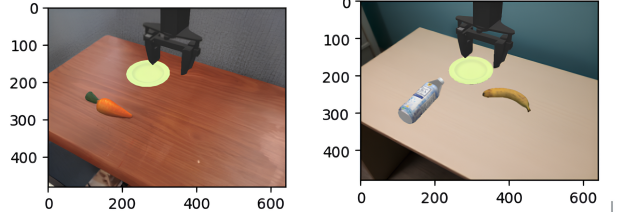


Figure 1. Environments for SR calculation

1. VLM Question-Based Evaluation

Task 1. Create a set of 5–8 questions for the VLM to assess its understanding of the scene, physical properties, spatial relations, etc. Choose 1–2 tasks from the repo and evaluate the model's answers based on the first image/video from the environment, compute success rate (SR).

As the VLM, **SmolVLM** was chosen, as suggested in the assignment. Based on the requirements of the task, the environments *PutCarrotOnPlateInScene-v1* and *PutOnPlateInScene25MultiCarrot-v1* were selected (Fig. 1).

The questions were designed to evaluate not only *how well the model can describe the environment*, but also *how effectively it understands the ongoing processes within it*. Five questions focused on environmental description:

- What objects do you see?
- What is the table made of?
- What is on the table?
- What can you eat on the table?
- What is closest to the edge of the table?

In addition, one question targeted the understanding of the ongoing action:

^{*}Equal contribution ¹Irkutsk State University. Correspondence to: Volovikov Georgiy <yyy>.

- What is most likely to fall off a table?

The model should be validated on these questions, and the Success Rate (SR) is to be calculated accordingly.

Metrics selections

Success Rate (SR) is a metric that measures the ratio of solved tasks to the total number of tasks. However, for VLMs this metric is not always directly applicable, since model outputs may be semantically correct while differing in structure from the expected answers. Therefore, an alternative metric is required to determine task correctness.

Hypothesis 1. To compute SR, it is preferable to use soft quality metrics such as BERTScore or the LLM-as-a-judge approach, since the model may produce semantically correct answers with a completely different structure.

Experimental setup. Two initial scene frames were prepared (random seed 42), as shown in Figure 1. Reference answers were prepared for each scene and compared using BERTScore. As an LLM-as-a-judge, the same model was employed under the assumption that, when shown the correct answer, it would be able to evaluate its own response.

Results. On average, BERTScore achieved 80% on the F1 score. It could be assumed that answers with scores above this threshold are correct. However, closer inspection showed that phrases such as *something beige* were scored as more similar to *wood* than the exact term *wood* itself. This suggests that BERTScore does not adequately address the task, likely due to the short length of the responses. The LLM-as-a-judge approach also failed, as it consistently classified answers as incorrect, even when the model output

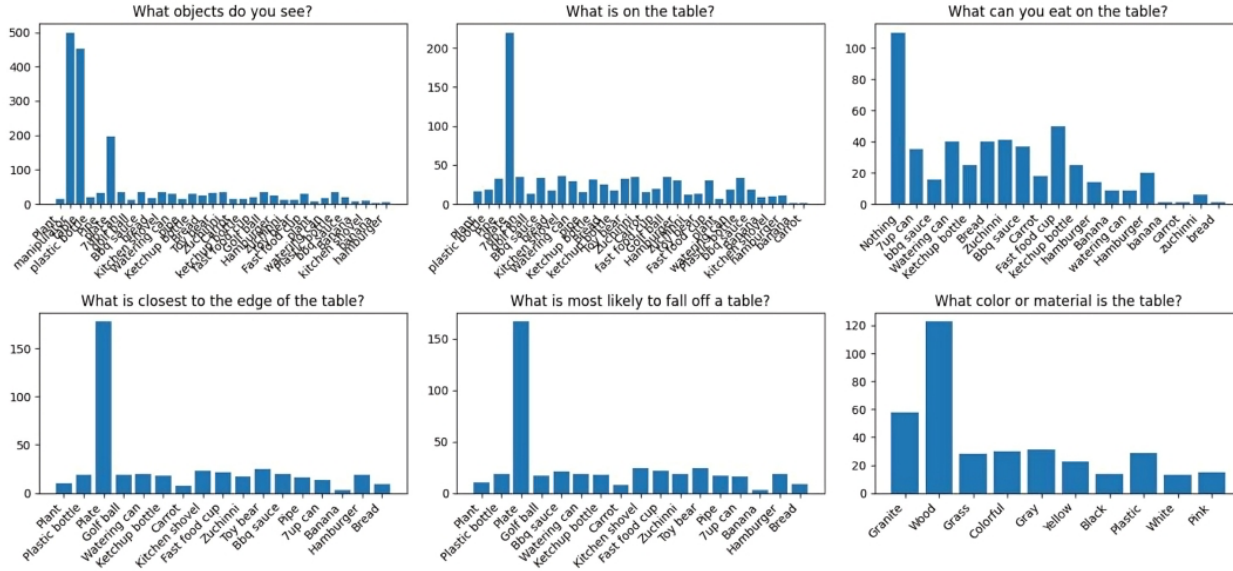


Figure 2. Enter Caption

Question	Predicted answer	True answer	BertScore (f1)	LLM-as-a-judge (smolVLM)	keywords
What objects do you see?	Bottle, plate, banana, and a microscope.	I see a table, a plate, a banana, a bottle and a manipulator.	True	False	False
What is the table made of?	Wood.	Table made of something beige.	True	False	False
What is on the table?	Bottle, plate, and bananas	There is a banana, a bottle and a plate on the table.	True	False	True
What can you eat on the table?	Banana.	I can eat a banana.	True	False	True
What is closest to the edge of the table?	Bottle.	A bottle	False	False	True
What is most likely to fall off a table?	Banana.	A bottle	False	False	False

Table 1. Charts showing which keywords in which questions she guesses the least often.

matched the reference exactly. It is possible that using a linguistic model embedded within SmolVLM instead of BERT could yield more reliable embeddings and results. The examples showed in the table 1.

Hypothesis 2. Since semantic evaluation did not yield satisfactory results, stricter approaches were tested. BLEU and similar statistical metrics were not suitable due to their heavy reliance on output structure. Instead, a keyword-based approach was introduced.

Experimental setup. The setup was identical to Hypothe-

sis 1, but instead of full reference answers, keyword sets were prepared for each question. During evaluation, model responses were checked for the presence of the required keywords at any position in the output.

Results. This keyword-based method of computing SR produced more satisfactory results, yielding an average of 75%. The examples showed in the table 1.

Scene Understanding Evaluation

SR was initially computed using only a single instance of each environment, which is not representative. When reloading the environment, new objects may appear on the table, and the surrounding context may change. Therefore, SR must be evaluated multiple times across diverse environment variations.

Experimental setup. Generative reference answers were created for the questions, depending on the generated environment. All answers could be extracted directly from the environment, except for those related to the relative position of objects with respect to the table edge. For these, distances from each object to the table edge were computed using their vector positions. The environment was instantiated 250 times, and model responses were collected for each run.

Results. The final SR achieved under this setup was 11.17%. Detailed statistics on the correctness of answers across questions are shown in Figure 2.

2. Zero-Shot Evaluation

Task 2. Evaluate the zero-shot capabilities of the VLM in the *PutOnPlateInScene25MultiCarrot-v1* environment by assessing its accuracy in object detection and computing the Success Rate (SR).

Object Recognition in the Scene

The next step was to evaluate the model’s ability to recognize objects within the scene.

Experimental setup. The setup was the same as in Hypothesis 2, with the target scene specified and a single question formulated, for which keywords were generated. The scene was instantiated 250 times, and model responses were evaluated accordingly.

Results. The final SR was 4%, and statistics of correct object recognition are shown in Figure 3. This result is significantly lower compared to previous experiments. A likely explanation is that, while the model is aware of the types of objects present in the scene, it does not necessarily know their exact names, which are extracted directly from the environment. This supports the earlier motivation to employ soft quality metrics for SR computation.

3. Supervised Fine-Tuning

Task-3. The initial quality is expected to be unsatisfactory; therefore, it is improved through supervised fine-tuning (SFT). In the *PutOnPlateInScene25MultiCarrot-v1* environment, a QA dataset is collected. Each sample consists of a scene image, a question (“What objects are in the scene?”),

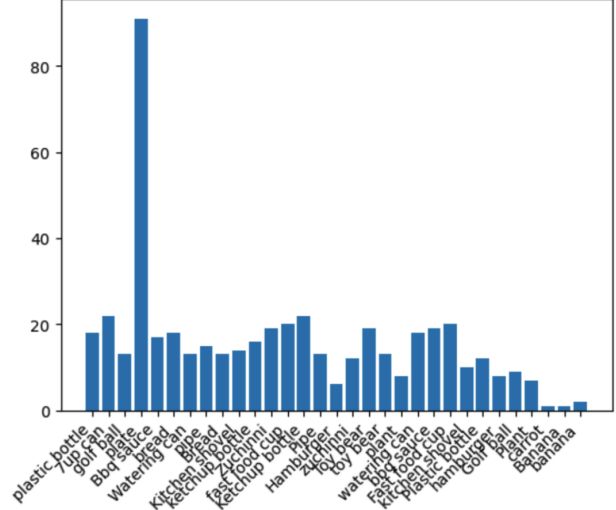


Figure 3. Object recognition statistics

and the corresponding answer (a list of objects).

SFT was intended to address the problem of SR calculation and to teach the model the correct naming of objects in the scene. Furthermore, training on an extended dataset, collected using multiple questions across different environments, was considered. It is assumed that training only on the initial dataset would not provide new capabilities or deeper understanding, but merely enable the model to correctly name objects for the SR calculation method. In contrast, training on the extended dataset, which includes questions requiring an understanding of the scene dynamics, could improve the model’s comprehension.

Hypothesis 3. SFT should resolve the SR calculation issue

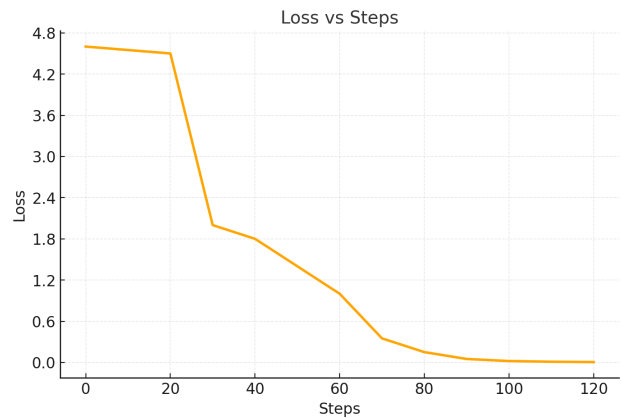


Figure 4. Loss on SFT task

and ensure that the model learns the correct naming of objects in the scene.

Experimental setup. The model was trained in Google Colab or Kaggle using Python 3.11–3.12. Training was performed with TRL, using 4-bit quantization via BitsAndBytes and LoRA. The training parameters were: batch size = 1, gradient accumulation steps = 4.

Results. All quality metrics during training were logged in Comet ML. The loss function trend over the course of training is shown in Figure 4.

3.1. Step 5: VLA Execution and State Prediction

From the VLA execution, the VLM is prompted every n simulation steps to describe the current action and state, e.g., “object not grasped”, “object grasped”, “object moving”, “object in target position”. The accuracy of these state predictions is evaluated in a zero-shot setting across three tasks. Techniques to improve zero-shot performance are also explored.

What did not work. The Octo model was selected as the VLA, since an interface for it was already available in the simplified environment provided in the assignment. Several configurations were attempted:

- Python 3.12 with numpy==2.0.0, numpy==1.26.4, and jax==0.4.20, jax==0.7.4
- Python 3.11 with numpy==2.0.0, numpy==1.26.4, and jax==0.4.20, jax==0.4.23, jax==0.4.33, flax==0.5.3, flax==0.6.4, flax==0.7.5, etc.

None of these configurations successfully executed.

Unverified hypotheses. Hypothesis 4. If SFT is applied on the extended dataset with the proposed questions, the model will not only learn object names but also achieve better scene understanding.